Code Anaylsis:

a. Name of two GenAI programs: Claude and Google Gemini

b. How you accessed them (e.g., Google query, CoPilot on my computer)
Claude was accessed through the Claude website on my computer and Gemini was accessed through the Google Gemini website on my computer.

c. Prompt used. (You must use the same prompt for both GenAIs.)

Keep the functionality and code of the program i give you but also add this functionality
Write the solutions for each puzzle file to a single file named solutions.txt.
• For each puzzle, the solutions.txt file should include the following:
o A single line: “Puzzle: x”, where x is the puzzle file name (e.g., puzzle1.txt).
o If no solutions are found, a single line: “No solution found.”
o If solutions are found, it should include all solutions followed by a single line: “Solutions found: n”, where n is the number of solutions found.
o The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.

(Prompt was given along with assignment 5 code received)

d. Generated code from both GenAIs.
Claude:

```
/*
 Program name: EECS 348 Assignment 5 (revision of Assignment 4 Sudoku Solver)
 Description: Recursive, object-oriented backtracking Sudoku solver that finds ALL solutions
 Inputs: puzzle1.txt - puzzle5.txt (81 tokens each, digits 1-9 or "_" for empty), or filenames given as arguments
 Outputs: solution1.txt - solution5.txt (every solution in the input format, then "Solutions found: n",
          or the single line "No solution found.") plus a short summary on the console,
          and one combined solutions.txt covering every puzzle (added by Josselyn with assistance from Claude)
 Collaborators: Eric Chen (original Assignment 4 author), Kimi K2.5 | Thinking (generated the original code)
```

```
 Other sources: Claude (Anthropic) helped with the revisions marked
"edited by Josselyn" below
 Original author: Eric Chen
 Revised by: Josselyn Bui
 Revision date: 10/29/2026
 Notes: Compile on the Cycle servers with: g++ -std=c++17 -o
Assignment5.exe Assignment5.cpp
*/

#include <array>    // fixed-size 9x9 grid type (original code)
#include <cctype>   // std::isdigit for checking tokens (original code)
#include <fstream>  // std::ifstream to read puzzles; std::ofstream to
write solutions (ofstream added by Josselyn)
#include <iostream> // std::cout / std::cerr for console messages
(original code)
#include <string>   // std::string for filenames and tokens (original
code)
#include <vector>   // std::vector for the list of filenames (original
code)

class Sudoku { // class holding the puzzle data and the recursive solver
(original structure, Eric Chen / Kimi)
public:
   // read(): loads one puzzle file. Returns false only for an unopenable
file or bad format.
   // edited by Josselyn with assistance from Claude: conflicting clues no
longer make read() fail
   bool read(const std::string& filename) {
       name = filename;                          // remember the file name
for messages (original)
       std::ifstream in(filename);               // open the puzzle file
(original)
       if (!in) {                                // file could not be
opened (original)
           std::cerr << "Could not open " << filename << "\n"; // tell the
user (original)
           return false;                         // main counts this as a
failed file (original)
       }
```

```
	grid.fill({});                                  // clear every cell to 0
(original)
	std::string tok;                                // holds one
whitespace-separated token (original)
	int count = 0;                                  // how many tokens we
have stored so far (original)

	while (in >> tok) {                             // read tokens until end
of file (original)
	    if (count >= 81) return false;              // more than 81 tokens
means a bad puzzle (original)
	    int value = 0;                              // value for this cell, 0
means empty (original)
	    if (tok == "_") {                           // underscore marks an
empty cell (original)
	        value = 0;                              // keep it empty
(original)
	    } else if (tok.size() == 1 && std::isdigit(static_cast<unsigned
char>(tok[0]))) { // single digit (original)
	        value = tok[0] - '0';                   // convert char to int
(original)
	        if (value < 1 || value > 9) return false; // 0 is not a
legal clue (original)
	    } else {
	        return false;                           // anything else is a
format error (original)
	    }
	    grid[count / 9][count % 9] = value;         // map linear index to
row and column (original)
	    ++count;                                    // move to the next cell
(original)
	}

	if (count != 81) return false;                  // a puzzle needs exactly
81 cells (original)

	// edited by Josselyn with assistance from Claude: record whether
the clues conflict instead of failing the read
	consistent = buildMasks();                      // false if any clue
repeats in a row, column or box
```

```
        return true;                                // file was readable and
well formed
    }

    // run(): solves the puzzle and writes solutionN.txt
    // edited by Josselyn with assistance from Claude: writes to a file,
uses the required messages
    // added by Josselyn with assistance from Claude: also writes this
puzzle's results to the shared solutions.txt stream
    void run(int puzzleNumber, std::ostream& combined) {
        allOut = &combined;                         // solve() uses this to
copy each solution into solutions.txt
        combined << "Puzzle: " << baseName() << "\n"; // start this
puzzle's section in solutions.txt
        std::string outName = "solution" + std::to_string(puzzleNumber) +
".txt"; // e.g. solution3.txt
        std::ofstream out(outName);                 // open the solution file
for writing
        if (!out) {                                 // could not create the
file
            std::cerr << "Could not create " << outName << "\n"; // tell
the user
            // no return here: the puzzle is still solved so solutions.txt
stays complete
        }

        solutions = 0;                              // reset the counter for
this puzzle

        if (!consistent) {                          // clues already break
the rules
            out << "No solution found.\n";          // required single line
            combined << "No solution found.\n\n"; // same result in
solutions.txt
            std::cout << name << ": No solution found.\n"; // console
summary
            return;                                 // skip the search
        }
```

```
        solve(0, out);                              // start recursion at
cell index 0

        if (solutions == 0) {                       // search finished with
nothing found
            out << "No solution found.\n";          // required single line
            combined << "No solution found.\n\n"; // same result in
solutions.txt
        } else {
            out << "Solutions found: " << solutions << "\n"; // required
final line
            combined << "\nSolutions found: " << solutions << "\n\n"; //
same count in solutions.txt
        }
        if (out) std::cout << name << ": " << solutions << " solution(s)
written to " << outName << "\n"; // console summary
        else std::cout << name << ": " << solutions << " solution(s) found
(" << outName << " not written)\n"; // file failed
    }

    // reportUnreadable(): lists an unopenable or badly formatted puzzle in
solutions.txt as having no solution
    // added by Josselyn with assistance from Claude
    void reportUnreadable(std::ostream& combined) const {
        combined << "Puzzle: " << baseName() << "\n"; // start this
puzzle's section
        combined << "No solution found.\n\n";        // nothing could be
solved
    }

private:
    std::string name;                               // current puzzle file
name (original)
    std::array<std::array<int, 9>, 9> grid{};      // the 9x9 grid, 0 =
empty (original)
    bool consistent = true;                         // false if the starting
clues conflict (original)
    int solutions = 0;                              // number of complete
solutions found (original)
```

```
    std::ostream* allOut = nullptr;                // added by Josselyn:
shared solutions.txt stream (set in run)

    // baseName(): file name without any folder path, e.g. puzzle1.txt
    // added by Josselyn with assistance from Claude
    std::string baseName() const {
        std::size_t slash = name.find_last_of("/\\"); // last folder
separator, if any
        return (slash == std::string::npos) ? name : name.substr(slash +
1); // keep only the file name
    }

    // edited by Josselyn with assistance from Claude: bitmasks replace the
repeated scans and vectors
    int rowMask[9] = {};                           // bit v is set when
digit v is used in that row
    int colMask[9] = {};                           // bit v is set when
digit v is used in that column
    int boxMask[9] = {};                           // bit v is set when
digit v is used in that 3x3 box

    // buildMasks(): fills the masks from the clues, returns false on a
duplicate
    // edited by Josselyn with assistance from Claude: replaces
initialGridIsConsistent (one pass instead of nested loops)
    bool buildMasks() {
        for (int i = 0; i < 9; ++i) {              // reset all masks first
            rowMask[i] = colMask[i] = boxMask[i] = 0;
        }
        for (int r = 0; r < 9; ++r) {              // visit every row
            for (int c = 0; c < 9; ++c) {          // visit every column
                int v = grid[r][c];                // digit in this cell
                if (v == 0) continue;              // empty cells have no
clue to check
                int bit = 1 << v;                  // bit that represents
this digit
                int b = (r / 3) * 3 + (c / 3);     // which of the 9 boxes
this cell is in
                if ((rowMask[r] & bit) || (colMask[c] & bit) || (boxMask[b]
& bit)) {
```

```
                    return false;                   // digit already used in
row, column or box
                }
                rowMask[r] |= bit;                  // mark digit used in the
row
                colMask[c] |= bit;                  // mark digit used in the
column
                boxMask[b] |= bit;                  // mark digit used in the
box
            }
        }
        return true;                                // no conflicts found
    }

    // solve(): recursive backtracking from cell index pos (0..80); prints
each full solution to out
    // edited by Josselyn with assistance from Claude: takes a start
position and uses masks (original logic by Eric Chen / Kimi)
    void solve(int pos, std::ostream& out) {
        while (pos < 81 && grid[pos / 9][pos % 9] != 0) ++pos; // skip
cells that are already filled

        if (pos == 81) {                            // no empty cell left, so
the grid is a solution
            if (solutions > 0) out << "\n";         // blank line between
solutions
            if (allOut && solutions > 0) *allOut << "\n"; // same blank
line in solutions.txt
            ++solutions;                            // count this solution
            writeGrid(out);                         // write it in the input
format
            if (allOut) writeGrid(*allOut);         // also write it to
solutions.txt
            return;                                 // go back and look for
more
        }

        int r = pos / 9;                            // row of the empty cell
        int c = pos % 9;                            // column of the empty
cell
```

```
        int b = (r / 3) * 3 + (c / 3);            // box of the empty cell
        int used = rowMask[r] | colMask[c] | boxMask[b]; // digits that are
not allowed here

        for (int v = 1; v <= 9; ++v) {            // try each digit in
increasing order
            int bit = 1 << v;                     // bit for this digit
            if (used & bit) continue;             // illegal here, try the
next digit
            grid[r][c] = v;                       // place the guess
            rowMask[r] |= bit;                    // update the masks to
match
            colMask[c] |= bit;
            boxMask[b] |= bit;
            solve(pos + 1, out);                  // recurse to the next
cell
            grid[r][c] = 0;                       // undo the guess
(backtrack)
            rowMask[r] &= ~bit;                   // undo the mask updates
            colMask[c] &= ~bit;
            boxMask[b] &= ~bit;
        }
    }

    // writeGrid(): prints the grid as 9 lines of space-separated tokens,
"_" for empty
    // edited by Josselyn with assistance from Claude: writes to any
stream, no trailing spaces
    void writeGrid(std::ostream& out) const {
        for (int r = 0; r < 9; ++r) {             // one output line per
row
            for (int c = 0; c < 9; ++c) {         // one token per column
                if (c > 0) out << ' ';            // space only between
tokens
                if (grid[r][c] == 0) out << '_';  // empty cell
                else out << grid[r][c];           // digit
            }
            out << "\n";                          // end of row
        }
    }
```

```
};

// program entry point
// edited by Josselyn with assistance from Claude: defaults to
puzzle1-5.txt, passes puzzle number to run()
int main(int argc, char** argv) {
   std::vector<std::string> files;               // puzzle file names to
process (original)

   for (int i = 1; i < argc; ++i) {              // filenames may come
from the command line (original)
       files.emplace_back(argv[i]);
   }

   if (files.empty()) {                          // nothing given, so use
the assignment's file names
       for (int i = 1; i <= 5; ++i) {            // puzzle1.txt through
puzzle5.txt
           files.push_back("puzzle" + std::to_string(i) + ".txt");
       }
   }

   std::ofstream combined("solutions.txt");      // added by Josselyn with
assistance from Claude: one file for all puzzles
   if (!combined) {                              // could not create
solutions.txt
       std::cerr << "Could not create solutions.txt\n";
       return 1;                                 // the required output
cannot be written
   }

   int failedFiles = 0;                          // how many files were
unreadable or badly formatted (original)
   int number = 0;                               // puzzle number used in
solutionN.txt
   for (const std::string& filename : files) {   // process each puzzle
(original)
       ++number;                                 // 1 for the first file,
2 for the second, ...
```

```
        Sudoku sudoku;                              // fresh solver object
for each file (original)
        if (sudoku.read(filename)) {                // load and validate the
format (original)
            sudoku.run(number, combined);           // solve; write
solutionN.txt and solutions.txt
        } else {
            sudoku.reportUnreadable(combined);      // list it in
solutions.txt as "No solution found."
            ++failedFiles;                          // count it and keep
going with the rest (original)
        }
    }

    if (failedFiles > 0) {                          // report problems at the
end (original)
        std::cerr << "Finished with " << failedFiles
                  << " file(s) that had errors. Check the files for bad
characters or formatting.\n";
        return 1;                                   // non-zero exit means
something failed (original)
    }
    return 0;                                       // everything worked
(original)
}
```

Gemini

```
*
 Program name: EECS 348 Assignment 5 (revision of Assignment 4 Sudoku
Solver)
 Description: Recursive, object-oriented backtracking Sudoku solver that
finds ALL solutions
 Inputs: puzzle1.txt - puzzle5.txt (81 tokens each, digits 1-9 or "_" for
empty), or filenames given as arguments
 Outputs: solution1.txt - solution5.txt (every solution in the input
format, then "Solutions found: n",
          or the single line "No solution found.") plus a single
solutions.txt file containing all results,
```

```
          and a short summary on the console
 Collaborators: Eric Chen (original Assignment 4 author), Kimi K2.5 |
Thinking (generated the original code)
 Other sources: Claude (Anthropic) helped with the revisions marked
"edited by Josselyn" below
 Original author: Eric Chen
 Revised by: Josselyn Bui
 Revision date: 10/29/2026 (Updated with solutions.txt support)
 Notes: Compile on the Cycle servers with: g++ -std=c++17 -o
Assignment5.exe Assignment5.cpp
*/

#include <array>    // fixed-size 9x9 grid type (original code)
#include <cctype>   // std::isdigit for checking tokens (original code)
#include <fstream>  // std::ifstream to read puzzles; std::ofstream to
write solutions (ofstream added by Josselyn)
#include <iostream> // std::cout / std::cerr for console messages
(original code)
#include <string>   // std::string for filenames and tokens (original
code)
#include <vector>   // std::vector for the list of filenames (original
code)

class Sudoku { // class holding the puzzle data and the recursive solver
(original structure, Eric Chen / Kimi)
public:
   // read(): loads one puzzle file. Returns false only for an unopenable
file or bad format.
   // edited by Josselyn with assistance from Claude: conflicting clues no
longer make read() fail
   bool read(const std::string& filename) {
       name = filename;                          // remember the file name
for messages (original)
       std::ifstream in(filename);               // open the puzzle file
(original)
       if (!in) {                                // file could not be
opened (original)
           std::cerr << "Could not open " << filename << "\n"; // tell the
user (original)
```

```
            return false;                               // main counts this as a
failed file (original)
        }

        grid.fill({});                                  // clear every cell to 0
(original)
        std::string tok;                                // holds one
whitespace-separated token (original)
        int count = 0;                                  // how many tokens we
have stored so far (original)

        while (in >> tok) {                             // read tokens until end
of file (original)
            if (count >= 81) return false;              // more than 81 tokens
means a bad puzzle (original)
            int value = 0;                              // value for this cell, 0
means empty (original)
            if (tok == "_") {                           // underscore marks an
empty cell (original)
                value = 0;                              // keep it empty
(original)
            } else if (tok.size() == 1 && std::isdigit(static_cast<unsigned
char>(tok[0]))) { // single digit (original)
                value = tok[0] - '0';                   // convert char to int
(original)
                if (value < 1 || value > 9) return false; // 0 is not a
legal clue (original)
            } else {
                return false;                           // anything else is a
format error (original)
            }
            grid[count / 9][count % 9] = value;         // map linear index to
row and column (original)
            ++count;                                    // move to the next cell
(original)
        }

        if (count != 81) return false;                  // a puzzle needs exactly
81 cells (original)
```

```
       // edited by Josselyn with assistance from Claude: record whether
the clues conflict instead of failing the read
       consistent = buildMasks();                // false if any clue
repeats in a row, column or box
       return true;                              // file was readable and
well formed
   }

   // run(): solves the puzzle, writes solutionN.txt, and appends results
to solutions.txt
   // edited by Josselyn with assistance from Claude: writes to files,
uses the required messages
   void run(int puzzleNumber, std::ostream& allSolutionsOut) {
       std::string outName = "solution" + std::to_string(puzzleNumber) +
".txt"; // e.g. solution3.txt
       std::ofstream out(outName);               // open the solution file
for writing
       if (!out) {                               // could not create the
file
           std::cerr << "Could not create " << outName << "\n"; // tell
the user
           return;                               // nothing more we can do
for this puzzle
       }

       solutions = 0;                            // reset the counter for
this puzzle

       // Write puzzle header to solutions.txt
       if (allSolutionsOut) {
           allSolutionsOut << "Puzzle: " << baseFilename() << "\n";
       }

       if (!consistent) {                        // clues already break
the rules
           out << "No solution found.\n";        // required single line
           if (allSolutionsOut) {
               allSolutionsOut << "No solution found.\n\n";
           }
```

```
            std::cout << name << ": No solution found.\n"; // console
summary
            return;                                 // skip the search
        }

        solve(0, out, allSolutionsOut);             // start recursion at
cell index 0

        if (solutions == 0) {                       // search finished with
nothing found
            out << "No solution found.\n";          // required single line
            if (allSolutionsOut) {
                allSolutionsOut << "No solution found.\n";
            }
        } else {
            out << "Solutions found: " << solutions << "\n"; // required
final line
            if (allSolutionsOut) {
                allSolutionsOut << "Solutions found: " << solutions <<
"\n";
            }
        }
        if (allSolutionsOut) {
            allSolutionsOut << "\n";                // blank line between
puzzles in solutions.txt
        }
        std::cout << name << ": " << solutions << " solution(s) written to
" << outName << "\n"; // console summary
    }

private:
    std::string name;                               // current puzzle file
name (original)
    std::array<std::array<int, 9>, 9> grid{};       // the 9x9 grid, 0 =
empty (original)
    bool consistent = true;                         // false if the starting
clues conflict (original)
    int solutions = 0;                              // number of complete
solutions found (original)
```

```
    // Helper to get just the filename (e.g. puzzle1.txt) from path if
needed
    std::string baseFilename() const {
        std::size_t start = name.find_last_of("/\\");
        return (start == std::string::npos) ? name : name.substr(start +
1);
    }

    // edited by Josselyn with assistance from Claude: bitmasks replace the
repeated scans and vectors
    int rowMask[9] = {};                        // bit v is set when
digit v is used in that row
    int colMask[9] = {};                        // bit v is set when
digit v is used in that column
    int boxMask[9] = {};                        // bit v is set when
digit v is used in that 3x3 box

    // buildMasks(): fills the masks from the clues, returns false on a
duplicate
    // edited by Josselyn with assistance from Claude: replaces
initialGridIsConsistent (one pass instead of nested loops)
    bool buildMasks() {
        for (int i = 0; i < 9; ++i) {           // reset all masks first
            rowMask[i] = colMask[i] = boxMask[i] = 0;
        }
        for (int r = 0; r < 9; ++r) {           // visit every row
            for (int c = 0; c < 9; ++c) {       // visit every column
                int v = grid[r][c];             // digit in this cell
                if (v == 0) continue;           // empty cells have no
clue to check
                int bit = 1 << v;               // bit that represents
this digit
                int b = (r / 3) * 3 + (c / 3);  // which of the 9 boxes
this cell is in
                if ((rowMask[r] & bit) || (colMask[c] & bit) || (boxMask[b]
& bit)) {
                    return false;               // digit already used in
row, column or box
                }
```

```
                rowMask[r] |= bit;                  // mark digit used in the
row
                colMask[c] |= bit;                  // mark digit used in the
column
                boxMask[b] |= bit;                  // mark digit used in the
box
            }
        }
        return true;                                // no conflicts found
    }

    // solve(): recursive backtracking from cell index pos (0..80); prints
each full solution to out and allSolutionsOut
    // edited by Josselyn with assistance from Claude: takes a start
position and uses masks (original logic by Eric Chen / Kimi)
    void solve(int pos, std::ostream& out, std::ostream& allSolutionsOut) {
        while (pos < 81 && grid[pos / 9][pos % 9] != 0) ++pos; // skip
cells that are already filled

        if (pos == 81) {                            // no empty cell left, so
the grid is a solution
            if (solutions > 0) {
                out << "\n";                        // blank line between
solutions in solutionN.txt
                allSolutionsOut << "\n";            // blank line between
solutions in solutions.txt
            }
            ++solutions;                            // count this solution
            writeGrid(out);                         // write it in the input
format
            writeGrid(allSolutionsOut);             // write it to
solutions.txt as well
            return;                                 // go back and look for
more
        }

        int r = pos / 9;                            // row of the empty cell
        int c = pos % 9;                            // column of the empty
cell
        int b = (r / 3) * 3 + (c / 3);              // box of the empty cell
```

```
        int used = rowMask[r] | colMask[c] | boxMask[b]; // digits that are
not allowed here

        for (int v = 1; v <= 9; ++v) {            // try each digit in
increasing order
            int bit = 1 << v;                     // bit for this digit
            if (used & bit) continue;             // illegal here, try the
next digit
            grid[r][c] = v;                       // place the guess
            rowMask[r] |= bit;                    // update the masks to
match
            colMask[c] |= bit;
            boxMask[b] |= bit;
            solve(pos + 1, out, allSolutionsOut); // recurse to the next
cell
            grid[r][c] = 0;                       // undo the guess
(backtrack)
            rowMask[r] &= ~bit;                   // undo the mask updates
            colMask[c] &= ~bit;
            boxMask[b] &= ~bit;
        }
    }

    // writeGrid(): prints the grid as 9 lines of space-separated tokens,
"_" for empty
    // edited by Josselyn with assistance from Claude: writes to any
stream, no trailing spaces
    void writeGrid(std::ostream& out) const {
        for (int r = 0; r < 9; ++r) {             // one output line per
row
            for (int c = 0; c < 9; ++c) {         // one token per column
                if (c > 0) out << ' ';            // space only between
tokens
                if (grid[r][c] == 0) out << '_';  // empty cell
                else out << grid[r][c];           // digit
            }
            out << "\n";                          // end of row
        }
    }
};
```

```
// program entry point
// edited by Josselyn with assistance from Claude: defaults to
puzzle1-5.txt, passes puzzle number to run()
int main(int argc, char** argv) {
   std::vector<std::string> files;              // puzzle file names to
process (original)

   for (int i = 1; i < argc; ++i) {             // filenames may come
from the command line (original)
       files.emplace_back(argv[i]);
   }

   if (files.empty()) {                         // nothing given, so use
the assignment's file names
       for (int i = 1; i <= 5; ++i) {           // puzzle1.txt through
puzzle5.txt
           files.push_back("puzzle" + std::to_string(i) + ".txt");
       }
   }

   std::ofstream allSolutionsFile("solutions.txt"); // Open single
combined output file
   if (!allSolutionsFile) {
       std::cerr << "Could not create solutions.txt\n";
   }

   int failedFiles = 0;                         // how many files were
unreadable or badly formatted (original)
   int number = 0;                              // puzzle number used in
solutionN.txt
   for (const std::string& filename : files) {  // process each puzzle
(original)
       ++number;                                // 1 for the first file,
2 for the second, ...
       Sudoku sudoku;                           // fresh solver object
for each file (original)
       if (sudoku.read(filename)) {             // load and validate the
format (original)
```

```
            sudoku.run(number, allSolutionsFile); // solve and write
solutionN.txt and solutions.txt
        } else {
            ++failedFiles;                        // count it and keep
going with the rest (original)
        }
    }

    allSolutionsFile.close();                     // Close combined
solutions file

    if (failedFiles > 0) {                        // report problems at the
end (original)
        std::cerr << "Finished with " << failedFiles
                  << " file(s) that had errors. Check the files for bad
characters or formatting.\n";
        return 1;                                 // non-zero exit means
something failed (original)
    }
    return 0;                                     // everything worked
(original)
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.

Correctness :

The program from the Claude generated code that performed as desired and expected. It kept all the functionality from Assignment 5 while also implementing the Assignment 6 functionality. It handles invalid inputs as well with appropriate error messages. The Gemini program performed the Assignment 6 functionality well with appropriate error handling, and kept all the Assignment 5 functionality. Both programs wrote to the solutions.txt file in a desirable format.

Execution Time:

Both Claude's and Gemini's programs have a worst-case time complexity of O(9^E), where E represents the number of empty cells in the Sudoku puzzle. Both programs use recursive backtracking, which involves attempting to place numbers 1 through 9 into

each empty cell until a valid solution is found. Since there can be up to 9 possible choices for each empty cell, the execution time can increase exponentially depending on the number of empty cells. Both programs also use bitmasks to check whether a number can be placed in a row, column, or box, allowing these checks to be completed in constant time, O(1). Reading and validating the Sudoku grid takes O(81) time, and writing each solution also takes O(81) time per output file. Overall, neither program has a significant advantage in execution time because they use essentially the same algorithm and have the same worst-case time complexity.

Space Complexity:
Both Claude's and Gemini's programs have a space complexity of O(E), where E represents the number of empty cells in the Sudoku puzzle. Both programs use a fixed 9×9 array to store the Sudoku grid and three bitmask arrays to keep track of the numbers already used in each row, column, and box. Since these arrays have a fixed size, they require constant space, O(1). However, both programs use recursive backtracking, which requires additional memory for each recursive function call. In the worst case, the recursion can reach a depth of E, resulting in O(E) space complexity. Both programs also write their solutions directly to output files instead of storing all the solutions in memory, which helps reduce memory usage. Overall, neither program has a significant advantage in space complexity because they use essentially the same approach.

Maintainability:
Both the generated codes have significant comments. The person whose code it was before used good comment practices and indicated specifically which parts they contributed. Both GenAi's added comments but did not specify the new portions contributed. Each program has good maintainability and would be easy to understand if returned to in the future.

f. Select one to use as the basis for this assignment and justify why you picked it.

There is not much difference between the programs as not much functionality need to be added between Assignment 5 functionality and Assignment 6. I am going to use the Claude generated code as the basis for this assignment because it has slightly better error handling making it more reliable.

g. Explain what you did to improve it in terms of correctness, execution time,

space complexity, and maintainability.

Most of the changes Claude made to the program focused on improving correctness rather than execution time or space complexity. Claude added the solutions.txt feature to meet the Assignment 6 requirements, allowing all puzzle solutions and their counts to be written into one file. Claude also improved error handling so that missing or incorrectly formatted files would be accounted for and the program could still write to solutions.txt if an individual solution file couldn't be created.

In terms of execution time, there weren't any major improvements since the program already used bitmasks and recursive backtracking. The time complexity remained O(9^E), although writing to two files slightly increased execution time. The space complexity also remained O(E), since solutions are written directly to files rather than stored in memory.

For maintainability, Claude added functions such as baseName() and reportUnreadable() to help organize the code. I then went through the program and labeled the new sections to distinguish Claude's changes from the original code. I also edited the inline comments to better explain the code in my own words and improve its readability.